AURA RETAIL OS

System Design and Prototype (Subtask 2)

Path B: Modular Hardware Platform

Prepared by – Prime Coders

Member Name	Student ID
Durgesh	202512051
Priyanshu	202512071
Moksh	202512120
Jay	202512082

Table of Contents

1. Project Overview

The Aura Retail OS project is about building a kiosk system for a fictional smart city called Zephyrus. The idea is that the same kiosk hardware gets deployed in very different places — a hospital, a metro station, a university, even a disaster relief zone — and each one needs to behave differently based on where it is and what it's selling.

This document is our Subtask 2 submission for Path B, which focuses on making the system modular from a hardware perspective. We have included partial source code for the patterns we've implemented so far, a working simulation you can run with Python 3, and an updated version of our class diagram.

Out of the patterns we planned in Subtask 1, we picked three to implement first. We went with these three because they cover the most fundamental parts of the system and were practical enough to get working in a simulation within the given time.

2. Design Patterns Implemented

Three design patterns have been implemented in this prototype. All three are directly mapped to Path B requirements from the project specification.

SN	Pattern	Filename	Requirement Addressed
1	Singleton	central_registry.py	Global config and kiosk registration
2	Abstract Factory	kiosk_factory.py	Compatible component creation per kiosk type
3	Adapter	payment_gateway.py	Multiple payment provider integration

3. Source Code

The following files form the partial implementation submitted for Subtask 2. The code is written in Python 3. No external libraries are required to run the simulation.

3.1 central_registry.py — Singleton Pattern

The CentralRegistry class uses the Singleton pattern to ensure that only one instance of the registry exists throughout the system. All kiosks register themselves to this single object. The static getInstance() method returns the existing instance if one already exists, or creates a new one otherwise.

Code -

```
# singleton - only one registry for whole system

class CentralRegistry:

  def __init__(self):
    self.kiosk_list = []
    self.conf = {
      "max_buy_limit": 5,
      "mode": "NORMAL"
    }
  }

  @classmethod def get_instance(cls):
    if cls._inst is None:
```

```
      cls._inst = CentralRegistry()
    }
    return cls._inst
  }

  def add_kiosk(self, k):
    self.kiosk_list.append(k)
    console.log('registered kiosk -> ' + k.kiosk_id)
  }

  def get_conf(self, key):
    return self.conf.get(key)
  }

  def set_conf(self, key, val):
    self.conf[key] = val
    print("config changed: " + key + " = " + str(val))
  }

  def get_all_kiosks(self):
    return self.kiosk_list
  }
}

CentralRegistry._inst = None
```

3.2 kiosk_factory.py — Abstract Factory Pattern

KioskFactory is an abstract base class that defines three factory methods: make_dispenser(), make_payment(), and make_inv_policy(). Each concrete factory class extends KioskFactory and returns components that are specifically suited to that kiosk type. This ensures that a pharmacy kiosk never accidentally gets a food kiosk's payment module or dispenser.

Code: -

```
# abstract factory - each kiosk type gets its own factory

class KioskFactory(ABC):
  make_dispenser()  { throw new Error('implement this') }
  make_payment()    { throw new Error('implement this') }
  make_inv_policy() { throw new Error('implement this') }
}

class PharmacyKioskFactory(KioskFactory):

  make_dispenser() {
    return {
      dtype: 'robotic_arm',
      give: function(item) {
        console.log('[pharma dispenser] giving out ' + item + ' via robotic arm')
      }
    }
  }

  make_payment() {
    return {
      ptype: 'card',
      pay: function(amt) {
        console.log('[pharma] card swiped for rs.' + amt)
      },
```

```
        refund: function(id) {
          console.log('[pharma] refund done for txn ' + id)
        }
      }
    }

    make_inv_policy() {
      return {
        pol: 'prescription_needed',
        verify: function(item) {
          console.log('[pharma] checking prescription for ' + item)
          return true
        }
      }
    }
}

class FoodKioskFactory(KioskFactory):

    make_dispenser() {
      return {
        dtype: 'spiral',
        give: function(item) {
          console.log('[food dispenser] spiral drop -> ' + item)
        }
      }
    }

    make_payment() {
      return {
        ptype: 'upi',
        pay: function(amt) {
          console.log('[food] upi payment done rs.' + amt)
        },
        refund: function(id) {
          console.log('[food] upi refund for ' + id)
        }
      }
    }

    make_inv_policy() {
      return {
        pol: 'normal',
        verify: function(item) {
          console.log('[food] stock ok for ' + item)
          return true
        }
      }
    }
}

class EmergencyKioskFactory(KioskFactory):

    make_dispenser() {
      return {
        dtype: 'conveyor',
        give: function(item) {
          console.log('[emergency dispenser] conveyor releasing ' + item)
        }
      }
    }

    make_payment() {
```

```
    return {
      ptype: 'free',
      pay: function(amt) {
        console.log('[emergency] no payment, item is free')
      },
      refund: function(id) {
        console.log('[emergency] nothing to refund')
      }
    }
  }

  make_inv_policy() {
    return {
      pol: 'emergency_override',
      verify: function(item) {
        console.log('[emergency] override active, releasing ' + item)
        return true
      }
    }
  }
}

from src.kiosk_factory import (KioskFactory, PharmacyKioskFactory,
          FoodKioskFactory, EmergencyKioskFactory)
```

3.3 payment_gateway.py — Adapter Pattern

Different payment providers expose incompatible APIs. The PaymentGateway class defines a common interface with do_payment() and do_refund() methods. UpiAdapter, CardAdapter, and WalletAdapter each wrap their respective third-party API and translate its calls into this common interface. Adding a new provider only requires writing one new adapter class without touching any existing code.

Code: -

```
# adapter pattern - different payment apis wrapped into one interface

class PaymentGateway(ABC):
  do_payment(amt)   { throw new Error('not implemented') }
  do_refund(txn_id)  { throw new Error('not implemented') }
}

# pretend third party apis below

class upi_api:
  send(upi_id, rupees) {
    return 'upi sent rs.' + rupees + ' to ' + upi_id
  }
  reverse(ref_no) {
    return 'upi reversed ref:' + ref_no
  }
}

class card_api:
  charge(card_no, amt) {
    return 'card ' + card_no + ' charged rs.' + amt
  }
  give_refund(receipt) {
    return 'card refund for receipt ' + receipt
  }
}
```

```
class wallet_api:
  deduct(w_id, amt) {
    return 'wallet ' + w_id + ' debited rs.' + amt
  }
  add_back(w_id, amt) {
    return 'wallet ' + w_id + ' credited rs.' + amt
  }
}

# actual adapters

class UpiAdapter(PaymentGateway):
  def __init__(self):
    super()
    this.api = new upi_api()
  }
  do_payment(amt) {
    let res = this.api.send('aura@upi', amt)
    console.log(res)
  }
  do_refund(txn_id) {
    let res = this.api.reverse(txn_id)
    console.log(res)
  }
}

class CardAdapter(PaymentGateway):
  def __init__(self):
    super()
    this.api = new card_api()
  }
  do_payment(amt) {
    let res = this.api.charge('xxxx-1234', amt)
    console.log(res)
  }
  do_refund(txn_id) {
    let res = this.api.give_refund(txn_id)
    console.log(res)
  }
}

class WalletAdapter(PaymentGateway):
  def __init__(self):
    super()
    this.api = new wallet_api()
  }
  do_payment(amt) {
    let res = this.api.deduct('usr_wallet_01', amt)
    console.log(res)
  }
  do_refund(txn_id) {
    let res = this.api.add_back('usr_wallet_01', 0)
    console.log(res)
  }
}

from src.payment_gateway import (PaymentGateway, UpiAdapter, CardAdapter, WalletAdapter)
```

4. Working Simulation

4.1 How to Run

To run the above code – Python 3.7+ must be installed (no other packages needed)

After copying the code, run "**python simulation.py**"

4.2 Simulation Code

The simulation creates one kiosk of each type using the Abstract Factory, verifies the Singleton registry, runs a few purchase transactions through the Facade interface, and demonstrates payment switching using the Adapter pattern.

Code: -

```
from src.central_registry import CentralRegistry
from src.kiosk_factory import (PharmacyKioskFactory,
    FoodKioskFactory,
    EmergencyKioskFactory)
from src.payment_gateway import (UpiAdapter,
    CardAdapter,
    WalletAdapter)
from src.kiosk_interface import KioskInterface
from src.i_product import SingleProduct

print("--- aura retail os simulation ---")
print("--- prime coders | it620 ---\n")

# singleton test
reg1 = CentralRegistry.get_instance()
reg2 = CentralRegistry.get_instance()
print("singleton check (should be true):", reg1 is reg2)

# abstract factory - kiosk creation
print("\n-- creating kiosks --")
pharma = KioskInterface('PHARMA-01', PharmacyKioskFactory())
food   = KioskInterface('FOOD-01',   FoodKioskFactory())
emrg   = KioskInterface('EMRG-01',   EmergencyKioskFactory())

# load some items
medicine = SingleProduct('Paracetamol', 25, 10)
let chips    = new SingleProduct('Chips', 20, 3)
let blanket  = new SingleProduct('Blanket', 0, 20)
pharma.load_item(medicine)
food.load_item(chips)
emrg.load_item(blanket)

# facade - buying through kiosk interface
print("\n-- purchases --")
pharma.buy('Paracetamol', 25)
food.buy('Chips', 20)
emrg.buy('Blanket', 0)

# adapter - same interface different payment providers
print("\n-- payment adapter test --")
upi    = UpiAdapter()
card   = CardAdapter()
wallet = WalletAdapter()
upi.do_payment(150)
card.do_payment(150)
wallet.do_payment(150)
upi.do_refund('TXN101')
```

```
# registry check
print("\n-- registry --")
all_kiosks = CentralRegistry.get_instance().get_all_kiosks()
print("total kiosks:", len(all_kiosks))
for (let k of all) { console.log(' ->', k.kiosk_id) }

print("\n--- done ---")
```

4.3 Expected Output

Running the simulation produces the following output:

```
PS D:\Durgesh\2nd Sem Labs\OOPS_Lab\primec-zephyrus> python simulation.py
aura retail os simulation --->
singleton check (should be true): True

creating kiosks -->
registered kiosk -> PHARMA-01
registered kiosk -> FOOD-01
registered kiosk -> EMRG-01

-- loading inventory --

[PHARMA-01] inventory:
  - Paracetamol | rs.25 | qty: 10
  [bundle] First Aid Kit | total: rs.40
     - Paracetamol | rs.25 | qty: 10
     - Bandage | rs.15 | qty: 5

[FOOD-01] inventory:
  - Chips | rs.20 | qty: 3
  - Water Bottle | rs.15 | qty: 0
  [bundle] Snack Combo | total: rs.50
     - Chips | rs.20 | qty: 3
     - Juice | rs.30 | qty: 2

[EMRG-01] inventory:
  - Blanket | rs.0 | qty: 20
  [bundle] Emergency Kit | total: rs.0
     - Blanket | rs.0 | qty: 20
     - Flashlight | rs.0 | qty: 15

purchase simulation -->

[PHARMA-01] buying: Paracetamol
[pharma] checking prescription for Paracetamol
[pharma] card swiped for rs.25
[pharma dispenser] giving out Paracetamol via robotic arm
done.

[FOOD-01] buying: Chips
[food] stock ok for Chips
[food] upi payment done rs.20
[food dispenser] spiral drop -> Chips
done.

[FOOD-01] buying: Water Bottle
Water Bottle is out of stock

[EMRG-01] buying: Blanket
[emergency] override active, releasing Blanket
[emergency] no payment, item is free
[emergency dispenser] conveyor releasing Blanket
done.

payment adapter test -->
```

5. Repository Structure

The GitHub repository is organized as follows:

```
aura-retail-os/
 src/
  central_registry.py   = Singleton
  kiosk_factory.py      = Abstract Factory
  payment_gateway.py    = Adapter
  i_product.py          = (planned: Composite)
  kiosk_interface.py    = (planned for later: Facade)
 diagrams/
  architecture.xml
  class-diagram.xml
 simulation.py
 README.md
```

File	Pattern	Purpose
src/central_registry.py	Singleton	Stores registered kiosks and system config
src/kiosk_factory.py	Abstract Factory	Creates dispenser, payment, policy per kiosk type
src/payment_gateway.py	Adapter	Unified interface for UPI, Card, Wallet APIs
simulation.py	All three	End-to-end working simulation

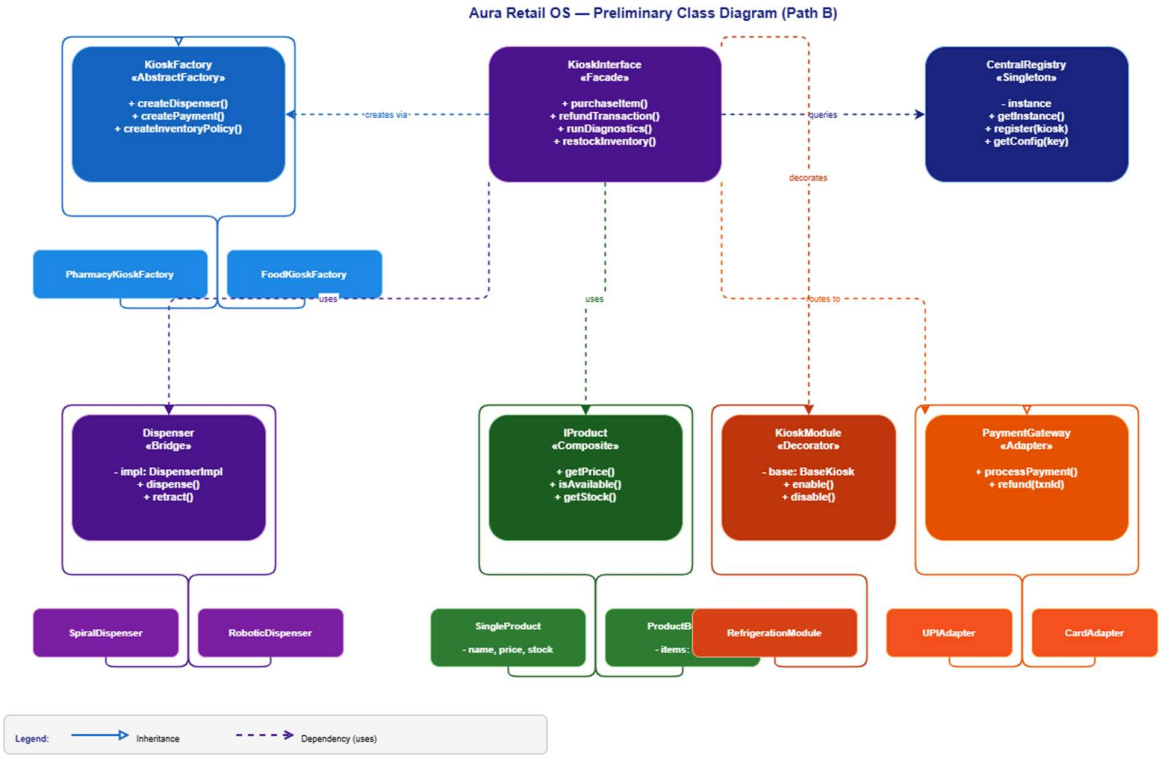
Github Repository Link - https://github.com/durgeshkhushlani/primec-zephyrus.git

6. Updated Class Diagram

The updated class diagram is available in the repository under diagrams/class-diagram.xml. It reflects the three implemented patterns and their class relationships as described in this document.

Key updates from the Subtask 1 preliminary diagram:

- CentralRegistry now shows the static getInstance() method and the _inst field explicitly
- KioskFactory shows all three concrete factories with their overridden methods
- PaymentGateway shows the three adapter classes and the third-party API classes they wrap


Aura Retail OS — Preliminary Class Diagram (Path B)
KioskFactory
«AbstractFactory»
+ createDispenser()
+ createPayment()
+ createInventoryPolicy()
creates via
KioskInterface
«Facade»
+ purchaseItem()
+ refundTransaction()
+ runDiagnostics()
+ restockInventory()
queries
CentralRegistry
«Singleton»
- instance
+ getInstance()
+ register(kiosk)
+ getConfig(key)
decorates
PharmacyKioskFactory
FoodKioskFactory
uses
uses
routes to
Dispenser
«Bridge»
- impl: DispenserImpl
+ dispense()
+ retract()
IProduct
«Composite»
+ getPrice()
+ isAvailable()
+ getStock()
KioskModule
«Decorator»
- base: BaseKiosk
+ enable()
+ disable()
PaymentGateway
«Adapter»
+ processPayment()
+ refund(txnId)
SpiralDispenser
RoboticDispenser
SingleProduct
- name, price, stock
ProductB
- items:
RefrigerationModule
UPIAdapter
CardAdapter
Legend:
Inheritance
Dependency (uses)